

AutoQuest: Automated Question Generation from Text Using Natural Language Processing

Arjun Karthikeya Nakka

Sai Rakshith A M S

Utham N

nakka.a@northeastern.edu
Northeastern University

avadhani.sa@northeastern.edu
Northeastern University

nagulapalli.u@northeastern.edu
Northeastern University

Abstract—AutoQuest is an innovative natural language processing system designed to automatically generate questions and answers from user-provided text. The system employs a two-stage architecture that processes textual content and produces educational assessment materials tailored to specific difficulty levels. The first component (M1) utilizes a fine-tuned T5-small model trained on a comprehensive dataset of context and questions with difficulty annotations given for each question. This model analyzes the input text and generates relevant questions based on the specified complexity level. The second component (M2) implements a Retrieval-Augmented Generation (RAG) approach to produce accurate, contextually appropriate answers to the generated questions. By combining these models, AutoQuest creates assessment materials that can be utilized in a variety of practical applications, including educational settings, content comprehension evaluation, and automated tutoring systems. The system’s ability to adapt to varying difficulty levels makes it particularly valuable for personalized learning applications and educational content development.

Index Terms—natural language processing, question generation, retrieval-augmented generation, educational technology, T5 model, difficulty levels, automated assessment

I. INTRODUCTION

It takes a lot of time and resources for educators and content producers to manually create high-quality evaluation questions. The procedure calls for pedagogical understanding, topic expertise, and a great deal of work to make sure the questions are suitable for the various learning goals and levels of difficulty. By offering an automated method for producing contextually relevant questions and answers from textual content, AutoQuest tackles this difficulty.

Major advances in text generation and comprehension have been made possible by recent developments in natural language processing (NLP). Transformer-based models may now produce logical, contextually relevant information and understand intricate relationships within text. These features are used by AutoQuest to evaluate user-provided documents and produce comprehension-testing questions at different levels of difficulty.

Although the question generating and response production processes are separated by the system’s two-stage architecture,

each component may be optimized specifically. With this method, AutoQuest can provide assessment resources that may be used in a variety of educational environments, such as tutoring platforms, self-assessment tools, and quizzes in the classroom.

II. LITERATURE SURVEY

The field of automatic question generation (AQG) has seen significant advancements in recent years, particularly with the integration of deep learning techniques. This section provides an overview of relevant research in this domain.

Pranav et al. [1] introduced Histobot, a question generation system employing deep learning techniques. Their research explores approaches to automatic question generation from historical data or documents, leveraging natural language processing capabilities to create meaningful questions. The system demonstrates how deep learning architectures can be effectively applied to generate questions that maintain coherence with the source material.

Yang et al. [2] presented an innovative approach to automatic question generation specifically for propositional logical equivalences. Their work addresses challenges in mathematics education, particularly in the context of increasing academic dishonesty during online learning. The system utilizes syntactic grammar and semantic attribute systems through top-down parsing and syntax tree transformations to produce logical equivalence problems. Their experimental results indicate that the generated questions achieve difficulty levels comparable to standard textbook problems, validating the effectiveness of their approach for educational applications.

The Web Tools for automatic Question Generation from code for Viva Preparation [3] focuses on developing tools that analyze programming code to generate relevant technical questions. This system helps computer science students prepare for oral examinations by automatically creating questions based on code structure, complexity, and patterns. Such tools represent an important application of AQG in technical education, addressing specific needs in computer

science pedagogy.

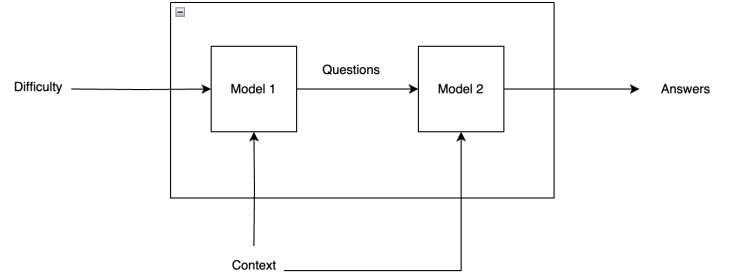
Lu and Lu [4] conducted a comprehensive survey of approaches to automatic question generation from 2019 to early 2021. Their analysis of recent papers highlights three key aspects: the purpose of question generation, generation methods, and evaluation techniques. The authors observed a trend toward semantic-based approaches and increasing adoption of Transformer-based models due to their efficiency advantages. Their survey also noted the persistent challenge of standardized evaluation metrics specifically designed for question generation tasks.

Lewis et al. [5] introduced a groundbreaking approach called Retrieval-Augmented Generation (RAG) for addressing knowledge-intensive natural language processing tasks. Their work explores the combination of parametric memory (pre-trained language models) with non-parametric memory (external knowledge sources) to enhance language generation capabilities. The authors developed RAG models where a pre-trained sequence-to-sequence model serves as parametric memory while a dense vector index of Wikipedia provides non-parametric memory. Their experiments demonstrated that RAG models generate more specific, diverse, and factual language than parametric-only models, setting state-of-the-art results on multiple open-domain question answering tasks. This work addresses key limitations of large language models, including their restricted ability to access and manipulate knowledge precisely, while also providing provenance for decisions and enabling world knowledge updates.

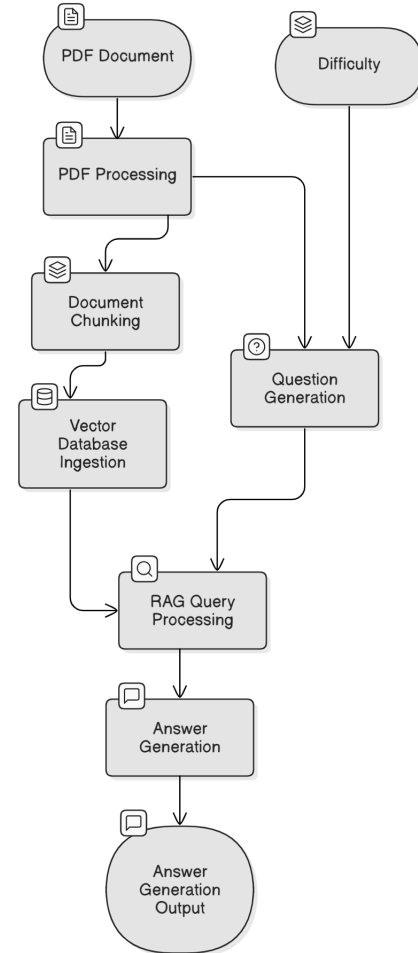
More recent work in the field of retrieval-augmented generation has expanded the scope and applications of RAG systems [6]. Researchers have continued to develop more sophisticated approaches that leverage external knowledge databases to augment large language models, addressing their inherent limitations such as hallucinations, knowledge cutoffs, and attribution challenges. These advancements have focused on improving both retrievers and retrieval fusion techniques, with applications spanning various natural language processing tasks including question answering, fact verification, and domain-specific information retrieval. The evolution of RAG systems demonstrates their increasing importance in knowledge-intensive NLP applications, particularly as demands for factual accuracy and attribution grow.

These studies collectively demonstrate the evolution of automatic question generation systems from rule-based approaches to more sophisticated deep learning architectures. The research shows particular promise in educational applications, where such systems can reduce the burden of manual question creation while maintaining question quality. Current challenges in the field include developing standardized evaluation metrics, improving semantic understanding, and adapting these technologies to specialized domains.

III. SYSTEM ARCHITECTURE(METHODS)



AutoQuest Data Flow Diagram



A. Overview

AutoQuest employs a two-stage architecture consisting of two primary models:

- **Model 1 (M1):** A fine-tuned T5-small model responsible for generating questions from the input text based on specified difficulty levels.
- **Model 2 (M2):** A Retrieval-Augmented Generation (RAG) model that produces accurate answers to the generated questions by referencing the original context.

Two inputs will be taken by the system: a PDF document with the text that will be used to produce the questions and a specification of the intended level of difficulty (easy, medium, or hard). A collection of question-answer pairs appropriate for evaluation makes up the output.

B. Question Generation Model (M1)

The T5-small model architecture, which has been refined on a specific dataset, is used in the question generating component. For text creation jobs, the T5 (Text-to-Text Transformer) model was used due to its adaptability and efficiency. Domain-specific fine-tuning improves the underlying model’s capacity to produce questions that are instructive, even though it still has great language understanding capabilities.

The fine-tuning dataset consists of triplets containing:

- Context passages from various domains
- Questions derived from these passages
- Corresponding answers
- Corresponding difficulty level annotations

The T5-small model is a transformer-based encoder-decoder architecture comprising approximately 60 million parameters. To enhance training efficiency, we froze the majority of the model’s layers and fine-tuned only the last two layers of both the encoder and decoder components.

Given that T5 supports in-context learning, we split the dataset into train, test and validation set. The train set is 72%, test is of 20% and validation is of 8%. We trained the model using structured prompts of the form:

generate question : {context} with difficulty {difficulty}

where difficulty can be one of easy, medium or hard. The model was trained using the AdamW optimizer with a learning rate of 5×10^{-5} for a total of 8 epochs.

For the question generation process, we utilized various decoding strategies to balance diversity and quality. These include:

- **Temperature sampling** with a temperature of 0.8 to control the randomness of predictions.
- **Top- p** and **Top- k** sampling to promote diverse yet coherent outputs.
- **Repetition penalty** to discourage the model from generating repetitive phrases.
- **n -gram blocking** to prevent repetition of longer phrases across the generated text.

To further minimize question repetition, we employed cosine similarity between generated questions, using a similarity threshold of 0.6. This ensured that the resulting questions were both distinct and contextually relevant.

C. Question Generation Alternative Model (M1)

Before transitioning to a text-to-text T5 transformer approach, we initially developed a hybrid architecture leveraging BERT as the encoder and GPT-2 as the decoder. This design was motivated by BERT’s strength in contextual understanding and GPT-2’s proficiency in generative tasks. To enhance the information flow between the encoder and decoder, we explicitly enabled and reinforced cross-attention mechanisms.

To optimize computational efficiency without significantly compromising performance, we froze the majority of BERT’s encoder layers, allowing only the final two layers to remain trainable. Additionally, we introduced custom special tokens to explicitly mark context boundaries, question prompts, and difficulty levels. Both BERT and GPT-2 token embedding matrices were resized to accommodate these new tokens, enabling fine-grained control during question generation.

The BLEU and ROUGE scores were used to evaluate the performance of the BERT-GPT2 hybrid model. Upon comparison, the T5-small model demonstrated superior performance on both metrics. This empirical advantage led us to adopt the T5 text-to-text model for the final pipeline implementation.

D. Answer Generation Model (M2)

The proposed Answer generation model consists of the RAG architecture. This architecture combines the benefits of retrieval-based and generation-based methods to produce accurate answers that remain faithful to the source material.

The proposed RAG model operates in two phases:

- **Retrieval Phase:** Identifies the most relevant passages from the input document that contain information needed to answer the generated questions.
- **Generation Phase:** Produces coherent, comprehensive answers based on the retrieved information while maintaining contextual accuracy.

This approach ensures that answers are both accurate and contextually appropriate, avoiding the hallucination problems sometimes associated with pure generation models.

The system begins by reading a PDF file using a utility that extracts raw text, section headers, and metadata such as the title and number of pages. The content is then processed into semantically meaningful chunks rather than arbitrary text blocks, with careful handling to preserve contextual structure like page numbers (inferred from markers) and section headers (extracted using pattern matching against known formats).

1) *Chunk Enrichment and Storage:* Each chunk is enriched by extracting top keywords using an enhanced TF-IDF-based method that supports multi-word phrases and filters out uninformative terms. These chunks are stored in a Milvus vector

database. The database is initialized with a custom collection schema that includes:

- A high-dimensional embedding vector (e.g., 768 dimensions)
- Textual content
- Scalar fields such as page number, section, document title, chunk position, and keywords

2) *Indexing Strategy*: To optimize retrieval, the collection is indexed using HNSW (Hierarchical Navigable Small World) indexing for the vector data, allowing for fast approximate nearest neighbor search using cosine similarity. Additionally, scalar indexes (e.g., sorted and inverted indexes) are created on fields like page number and section to enable hybrid and filtered queries.

3) *Hybrid Retrieval Mechanism*: When a user submits a question, the system performs hybrid retrieval to find the most relevant content. This combines:

- **Semantic search** using embeddings generated from the question
- **Lexical search** using BM25, a ranking function based on keyword frequency and distribution

4) *Re-ranking and Context Construction*: The retrieved chunks are then re-ranked using a cross-encoder model (such as ms-marco-MiniLM) that takes the question and each chunk as a pair and scores them based on relevance. The top-ranked chunks are selected and reformatted with metadata (e.g., section and page details) to form the final context.

5) *Answer Generation*: This context is passed as input to a generative language model (e.g., google/flan-t5-large). The system constructs a prompt that includes both the retrieved chunks and the question, instructing the model to answer strictly using the provided context. The model is configured with generation constraints such as:

- Beam search
- Temperature control
- N-gram repetition blocking

If the answer cannot be confidently formed from the context, the system is designed to clearly indicate the lack of sufficient information instead of making assumptions.

IV. DATASET

A. Dataset Creation and Characteristics

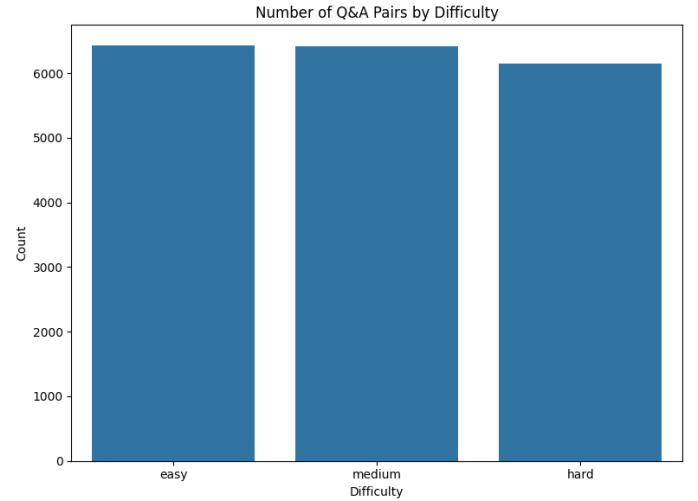
For the AutoQuest system, we developed comprehensive datasets to support both the question generation (M1) and answer generation (M2) components. The datasets were synthetically generated using the Gemini API, allowing us to create a large corpus of context-question-answer triplets across multiple difficulty levels without the high costs and

time constraints typically associated with human annotation.

The generation process involved providing the Gemini API with contextual passages and instructing it to generate questions of specific difficulty levels (easy, medium and hard) along with their corresponding answers. This approach enabled us to create a structured dataset where each context has multiple associated questions, systematically varying in complexity. In comparison to conventional human-annotated methods, we greatly decreased the resources needed by using large language models (LLMs) for the creation of synthetic data while preserving constant difficulty calibration across samples.

B. Question Generation Dataset (M1)

For the question generation model, we created a dataset with statistics summarized in Table I.



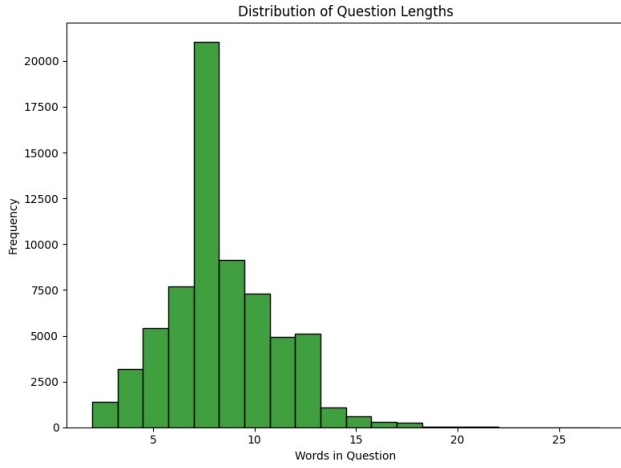
Metric	Value
Unique Contexts	6,327
Total Questions	18,981
Questions per Context	3
Scientific Topics Covered	100
Difficulty Levels	3 (Easy, Medium, Hard)

TABLE I
QUESTION GENERATION (M1) DATASET STATISTICS

This dataset structure supports the training of our T5-small model to recognize patterns between context and questions at varying difficulty levels. The diversity of topics ensures that the model can generalize across different domains, while the difficulty annotations enable the model to learn features associated with question complexity.

C. Answer Generation Dataset (M2)

For the Retrieval-Augmented Generation (RAG) answer model, we developed a separate dataset with statistics summarized in Table II.



Metric	Value
Unique Contexts with Q&A	67,501
Scientific Topics Covered	100
Context-Question-Answer Triplets	67,501

TABLE II
ANSWER GENERATION (M2) DATASET STATISTICS

This larger dataset provides the RAG model with sufficient examples to learn effective retrieval patterns and answer generation strategies. The comprehensive nature of this dataset enables the model to identify relevant context segments and generate answers that are both accurate and contextually appropriate.

D. Dataset Validation and Quality Control

To ensure dataset quality, we implemented **Random sampling and manual review of close to 450 different contexts** of generated content. This way of quality control measure helped maintain the integrity of our datasets and ensure that the trained models would produce high-quality outputs. The synthetic nature of our data generation process allowed us to rapidly iterate and refine the datasets based on validation results, resulting in improved training data quality.

V. EVALUATION AND RESULTS

A. Evaluation Metrics

The system was evaluated using both automated metrics and human assessment. Automated metrics included:

- BLEU and ROUGE scores for comparing generated questions and answers against reference examples
- Semantic similarity was calculated using cosine similarity between generated questions with a threshold of 60%.

Human evaluation involved us rating questions on relevance, clarity, and appropriateness for the specified difficulty level.

B. Performance Results

The performance results demonstrate a clear superiority of the T5 (Text-to-Text Transformer) model over the BERT-GPT2 Hybrid approach across all evaluation metrics. With an overall

Model	BLEU	ROUGE by Difficulty Level		
		Easy	Medium	Hard
T5 (Text-to-Text Transformer)	20.19	59.44	42.91	39.41
BERT-GPT2 Hybrid	5.58	35.19	28.51	30.81

TABLE III
COMPARATIVE PERFORMANCE OF QUESTION GENERATION MODELS

Model	BLEU	ROUGE
RAG	63	96.03

TABLE IV
COMPARATIVE PERFORMANCE OF RAG MODEL

BLEU score of 20.19, the T5 model significantly outperforms the BERT-GPT2 hybrid, which achieved only 5.58. This substantial difference indicates that the T5 architecture's unified text-to-text framework is more effective for question generation tasks than the encoder-decoder combination in the BERT-GPT2 model.

Looking at the ROUGE scores across difficulty levels, both models show a consistent pattern of declining performance as the complexity of the question increases, which aligns with expectations. The T5 model maintains its advantage across all difficulty categories, with particularly strong results for easy questions (59.44). Notably, the performance gap between the two models is most pronounced for easy questions, while narrowing slightly for medium and hard questions, suggesting that as the complexity of the question increases, the inherent difficulty of the task becomes a more significant factor than differences in model architecture.

VI. ADDITIONAL MODELS

For the question generation model, our research initially explored a BERT-BERT encoder-decoder architecture, where both the encoder and decoder were initialized using pre-trained BERT-base-uncased weights. This setup allowed us to take advantage of BERT's rich contextual representations learned during its pre-training phase. To balance training efficiency with model performance, we applied selective layer fine-tuning, freezing all layers except for the last two.

Although this method yielded promising early results, we noticed limitations in the fluency and diversity of the generated questions. Based on these observations, we shifted to a hybrid BERT-GPT2 architecture, pairing BERT as the encoder for its strong contextual understanding with GPT2 as the decoder, known for its robust text generation capabilities. This hybrid setup effectively combines the bidirectional context processing of BERT with the autoregressive language modeling strengths of GPT2—resulting in more fluent and diverse question generation, aligning better with the goals of our task.

The initial model used in the answer generation component was an encoder-based architecture, chosen for its efficiency in handling sequential data. However, after training and evaluating its performance, it became clear that the model was falling short—often failing to generate accurate responses when the required information wasn’t explicitly present in the input context. Recognizing this limitation, the team shifted focus to exploring alternative architectures that could better capture contextual dependencies.

Decoder-based models emerged as a strong candidate, given their success in various natural language processing tasks. We began experimenting with these models to improve the system’s ability to generate contextually relevant responses. By leveraging the self-attention mechanism of transformers, the model could dynamically focus on different parts of the input when generating text, leading to notable improvements.

Despite these advancements, a major challenge persisted—hallucination, where the model would confidently generate inaccurate or fabricated information. This highlighted the need for a more grounded approach. To address this, we refined our training methodology and introduced better data curation techniques. Ultimately, this progression led us to adopt a Retrieval-Augmented Generation (RAG) framework, which anchors the model’s output in actual retrieved content, significantly improving reliability and factual accuracy.

VII. DISCUSSION

The following challenges were faced during the course of this project:

- Finding an appropriate and large enough dataset to meet the project requirements, there was no particular dataset that had STEM questions and answers. So we had to generate a synthetic dataset using Gemini APIs.
- Model M1 in the initial stages threw out a lot of repetitive questions, we had to penalize it for duplicate questions and also had to use cosine similarity to filter duplicate questions and output as many unique questions as possible.
- While working on model M2, the major issue we faced was the efficiency of retriever, it was not very optimised, if the similarity score is set too high it ends up getting no chunks and if its too low, it ends up giving data which might not be very relevant, and also since we have a limited processing power, we had issues to work with large sized PDFs.
- The choosing of optimised chunk size and the overlap size in the vector database was another challenge we faced. We decided the current values after a lot of trial and error attempts(parameter tuning).
- As this project’s output was a subjective result, we could not find a particular metric to determine our models’ efficiency. We had to settle for manually checking some

responses and also BLEU and ROUGE scores.

VIII. FUTURE WORK

Future developments have been identified in several directions:

- Developing the model to manage multimodal inputs, such as schematics and pictures
- Adding domain-specific knowledge bases to improve the relevance of questions in specialist subjects
- The addition of explanation generating capabilities to promote learning in conjunction with evaluations
- Extend the core functionalities to accommodate other languages.
- Developing a more sophisticated difficulty calibration that takes into account many complexity dimensions

IX. CONCLUSION

AutoQuest demonstrates the potential of modern NLP techniques to address the challenges in the creation of educational content. By automating the generation of high-quality questions and answers at various difficulty levels, the system offers significant time savings for educators while maintaining the quality of the assessment. The two-stage architecture, combining a fine-tuned T5 model with a RAG approach, provides both the flexibility to generate diverse questions and the accuracy to produce reliable answers. As natural language processing capabilities continue to advance, systems like AutoQuest are poised to play an increasingly important role in educational technology and content development.

CONTRIBUTIONS

Arjun - Fine-tuned BERT-GPT2 model, PPT, Report, Front-end integration and dataset preparation.

Sai Rakshith - Fine-tuned T5 Model, PPT, Report, Frontend integration and dataset preparation.

Utham - RAG Model, PPT, Report, Frontend integration and dataset preparation

APPENDIX A

Prerequisites:

- Python 3.8 or higher
- Docker Desktop
- WSL 2 (Windows Subsystem for Linux 2)

Procedure:

• Setup the Vector Database:

- Once you have your environment setup, you need to download the installation script for Milvus Vector DB environment.

```
Invoke-WebRequest https://raw.githubusercontent.com/milvus-io/milvus/refs/heads/master/scripts/standalone_embed.bat
```



```
-OutFile standalone.bat
```

- Run the script to set up the Milvus environment by executing **standalone.bat** in your command line.
./standalone.bat start

- If you see the below message in the terminal your DB is up and running successfully.
Wait for Milvus starting...\nStart successfully.\nTo change the default Milvus configuration, edit user.yaml and restart the service.

- For more info look into the website: Install Milvus Standalone

Install the application requirements:

- You can find the requirements.txt file with the application code. Run the below command to install the requirements. You can find the requirements.txt file in f1 folder.
pip install -r requirements.txt
- If you have no errors proceed, else fix the errors.

Download the required models:

- Download the T5 model(model to be used for pipeline) at link and rename to **model1_t5.pt**
- Download the BERT-GPT2 model at link and rename to **model2_BERT-GPT2.pt**.
- Save this file in the model folder of the given code.

Run the code:

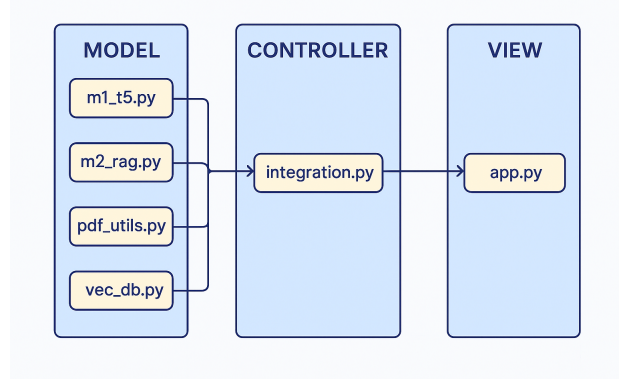
- To start the frontend of the application, you need to run the **app.py** file.
streamlit run app.py
- Open your web browser and navigate to http://localhost:8501 to access the application.
- Ensure that the Milvus server is running before accessing the frontend. If you encounter any issues, check the logs for troubleshooting.

The submitted ZIP file contains two main folders and a ReadMe.md file. The folders are organized as follows:

- **f1**: This folder includes all the core code required to run the application.
- **f2**: This folder contains supporting code files that were used during the development and fine-tuning of the components found in f1.

The datasets used for training are hosted externally on GitHub link. Access links are available in the References section of this report. To set up the environment and dependencies necessary to run the application, please refer above included

in the ZIP archive. For additional technical details, you can also visit the GitHub repository linked in the References.



The codebase follows an MVC (Model-View-Controller) architectural pattern as shown in above image:

• Model

- **m1_t5.py**: Contains the logic for question generation using a T5-based model.
- **m2_rag.py**: Implements the answer generation pipeline based on a Retrieval-Augmented Generation (RAG) framework.
- **pdf_utils.py**: Provides utilities for extracting content and metadata from PDF files.
- **vec_db.py**: Manages all operations related to the vector database, including setup, insertion, search, and deletion of data.

• Controller:

- **integration.py**: Serves as the central integration module that coordinates interactions between the various model components, encapsulating them within a single class for streamlined execution.

• View:

- **app.py**: Implements the user interface using Streamlit, enabling user interaction with the system in a web-based application format.

There is also a test.py file which contains the test cases written to test this application functionality. For testing purposes there is also a resources folder which contains a few sample pdfs. Also, there is a model folder in which the user is expected to put the model as instructed above.

The second file f2 consists of all the code we wrote during fine-tuning the LLMs and data generation. Namely:

- Bert based encoder-decoder model.
- Finetuned T5-small.
- Finetuned Sci-Bert for answer generation.
- RAG model
- Data generation with difficulty.
- Data generation without difficulty.
- Finetuned BERT-GPT2

REFERENCES

- [1] D. Pranav and V. R. Badri Prasad, "Histobot: Question Generation System Using Deep Learning Techniques," in RSP Science Hub In-

ternational Research Journal on Advanced Science Hub, vol. 5, no. 5, pp. 159-165, May 2023.

- [2] Y. Yang, X. Wang, H. Yu, and Z. Li, "Automatic question generation for propositional logical equivalences," arXiv preprint arXiv:2405.05513, May 2024.
- [3] H. Gaur, D. K. Tayal, and A. Jain, "ViQG: Web Tool for Automatic Question Generation from Code for Viva Preparation," in 2023 26th Conference of the Oriental COCODA.
- [4] C.-Y. Lu and S.-E. Lu, "A Survey of Approaches to Automatic Question Generation: from 2019 to Early 2021," in Proceedings of the 33rd Conference on Computational Linguistics and Speech Processing (ROCLING 2021), Taoyuan, Taiwan, Oct. 2021, pp. 151-162.
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," arXiv preprint arXiv:2005.11401, May 2020.
- [6] Z. Chen, C. Xu, D. Wang, Z. Huang, Y. Dou, X. Jiang, and J. Guo, "RuleRAG: Rule-Guided Retrieval-Augmented Generation with Language Models for Question Answering," arXiv preprint arXiv:2410.22353, Oct. 2024.